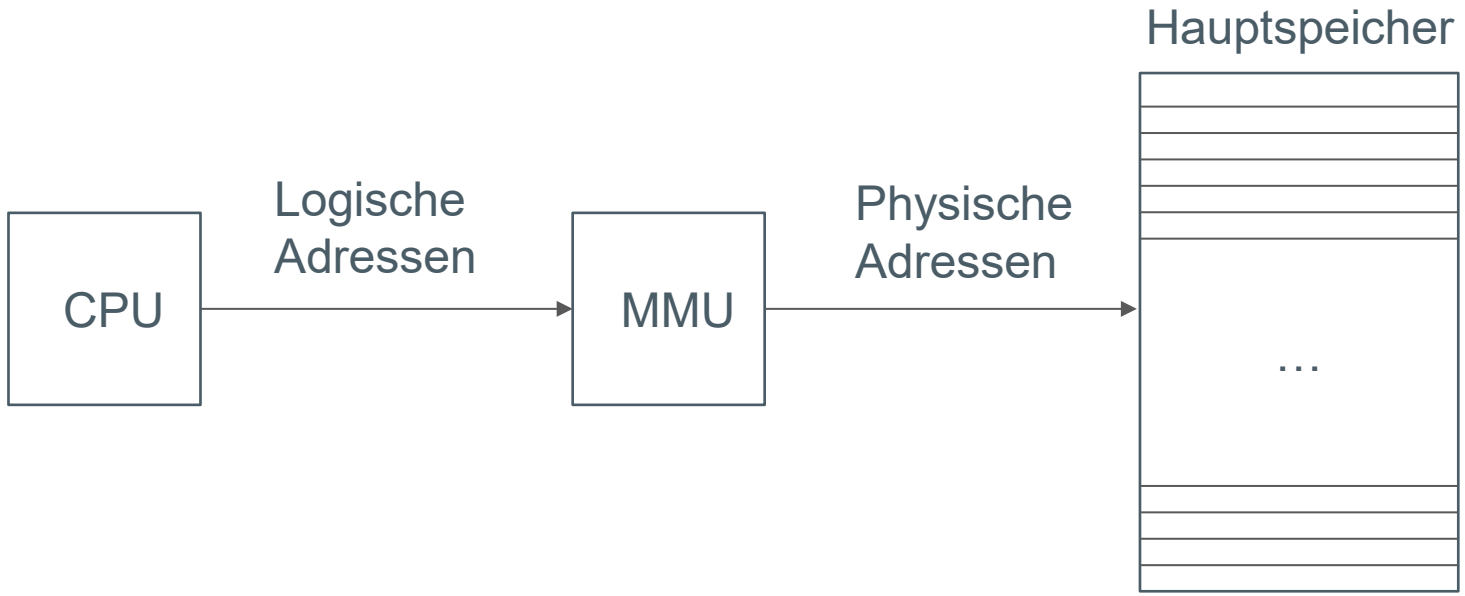
Agenda

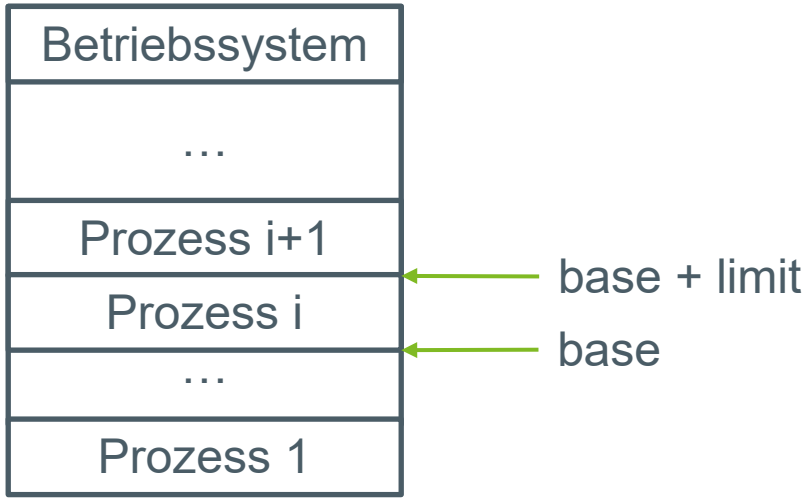
1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Memory Management Unit (MMU)

- Die CPU arbeitet mit logischen Adressen
- Die MMU ist ein Hardware-Gerät, das die logischen Adressen in physische Adressen im Hauptspeicher übersetzt.



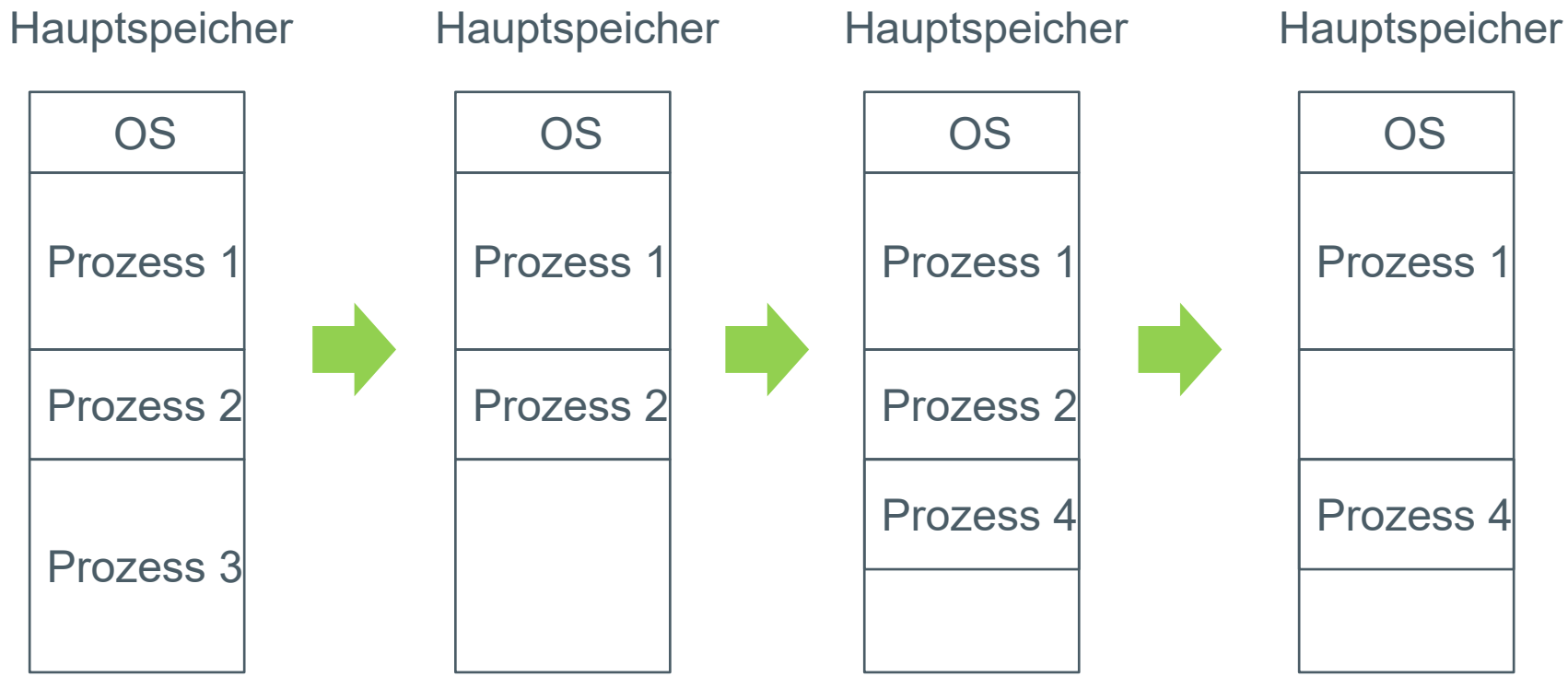
Herausforderungen bei der Speicherverwaltung



1. Ein Prozess darf nicht auf Speicher eines anderen Prozesses zugreifen
   - Dazu werden zwei Register (base und limit) im Prozesskontext verwaltet
   - Die Hardware überwacht beim Speicherzugriff, dass nur auf Speicher zwischen base und base+limit zugegriffen wird.

2. Der physische Speicher reicht evtl. nicht aus, um alle Prozesse gleichzeitig im Speicher zu halten.
   - Jeder Prozess hat einen **logischen Adressraum**, der zur Laufzeit durch die Memory Management Unit auf den physischen Speicher abgebildet wird.

Zusammenhängender Speicher

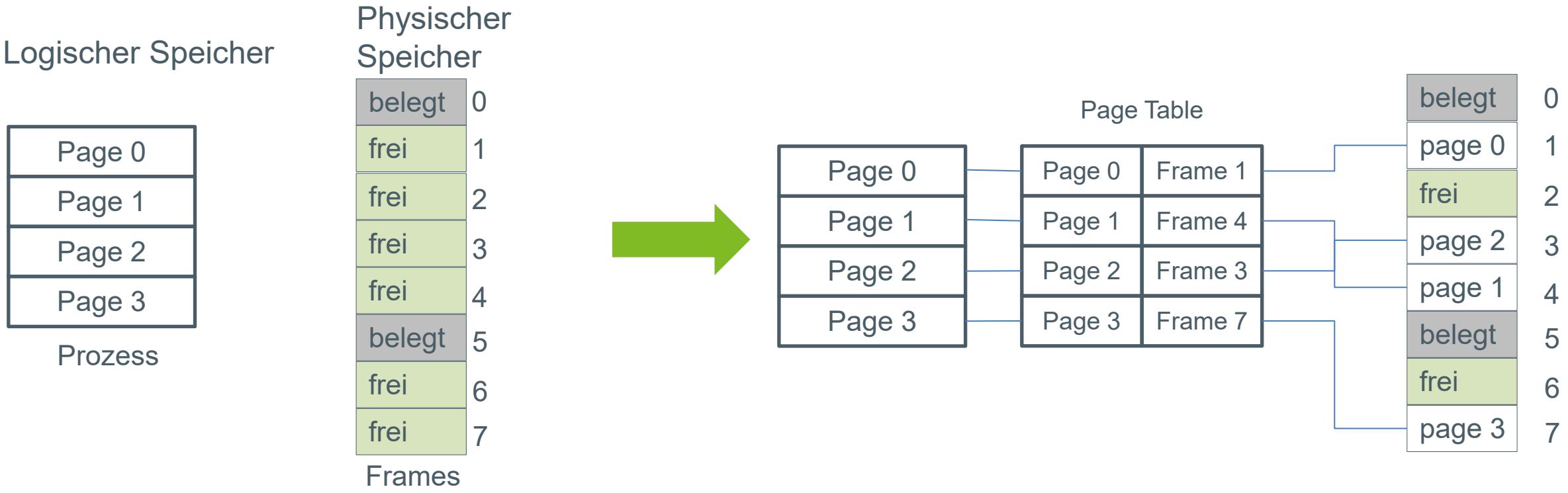


Externe Fragmentierung: Der physische Speicher ist in kleine, nicht zusammenhängende Blöcke zerteilt, die aber zusammen ausreichend groß sind, um die Prozesse zu laden.

Folge: Leistungsverlust!

Lösung: Paging

- Der physische Speicher wird in Blöcke bestimmter Größe aufgeteilt (Frames) – meistens 4KB
- Der logische Speicher wird in Blöcke gleicher Größe aufgeteilt (Pages).
- Jeder Prozess hat eine Page Table, die die Pages auf die Blöcke abbildet.



Fragmentierung

Bei Verwendung von Paging kann es nicht zu externer Fragmentierung kommen:

- Wenn ein Prozess Speicher anfordert, wird dieser in **Pages** aufgeteilt.
- Diese Seiten werden dann in **beliebigen freien Frames** im Hauptspeicher abgelegt.
- Da alle Frames gleich groß sind, spielt die Reihenfolge der Speicheranforderungen keine Rolle.
- Es gibt **keine Lücken** zwischen den Frames, die nicht genutzt werden können.

Interne Fragmentierung bedeutet, dass für einen Prozess ein Speicherbereich reserviert wird, der größer ist als der tatsächlich benötigte. Darauf hat Paging keinen Einfluss!
Folge: Verschwendung von Speicher!

Virtueller Speicher

Beobachtung 1:

- Der Code muss im Speicher sein, um ausgeführt zu werden, aber das gesamte Programm wird selten verwendet.
- Wenn das Programm nur teilweise in den Speicher geladen wird:
  - Programme sind nicht mehr durch die Grenzen des physischen Speichers beschränkt.
  - Jedes Programm benötigt weniger Speicherplatz während der Ausführung
    -> mehr Programme können gleichzeitig ausgeführt werden.
  - Weniger Ein- und Ausgabe ist erforderlich, um Programme in den Speicher zu laden
    -> jedes Benutzerprogramm läuft schneller.

Beobachtung 2:

- Viele Programme benutzen die gleichen Bibliotheken.
- Wenn dieser Code von mehreren benutzt werden kann, muss er nur einmal im physischen Speicher sein.

Beobachtung 3:

- Mit Paging alleine können wir keinen Prozess ausführen, der alleine den gesamten Speicher benötigt.

Demand Paging

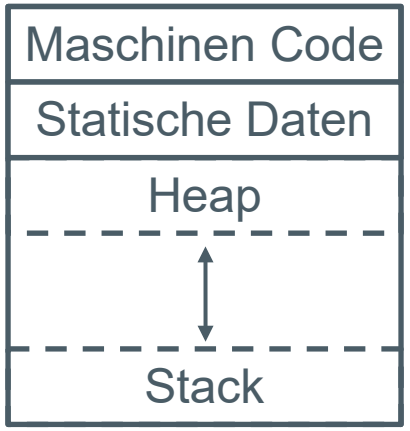
- **Demand Paging** ist eine Möglichkeit, virtuellen Speicher zu implementieren.
- Seiten werden erst dann geladen, wenn sie gebraucht werden.
- Vorteil?
  - Weniger I/O-Operationen bei Prozesswechsel
  - Benötigt weniger Speicher beim Start
  - Programm startet schneller

Algorithmus:

- Wird eine Seite benötigt, die bereits im Speicher ist (z.B. von anderem Prozess verwendet): Keine Aktion notwendig
- Sonst: Laden der Seite von der Festplatte und Suchen eines freien Frames

Aufteilung des Speichers eines Prozesses

Der Speicher, den ein Prozess belegt, ist im Allgemeinen wie folgt aufgeteilt (logische Sicht):

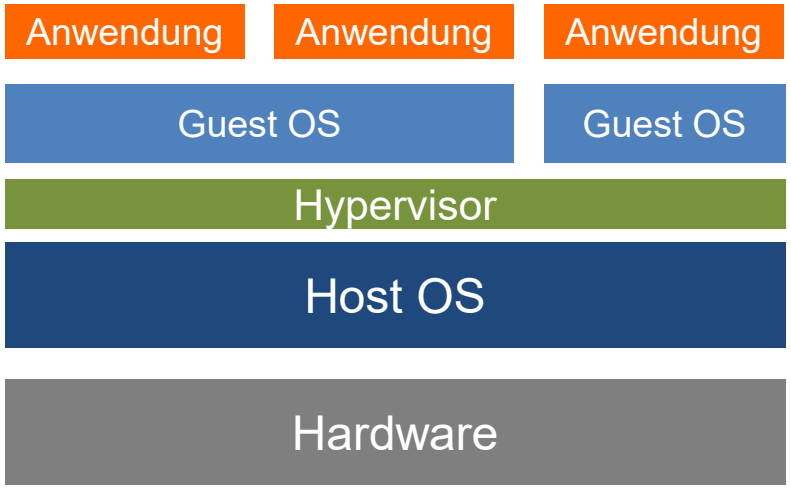


Heap wird benötigt für dynamischen Speicher (z.B. durch new in Java)
Stack wird benötigt für Methoden-/Funktionsaufrufe

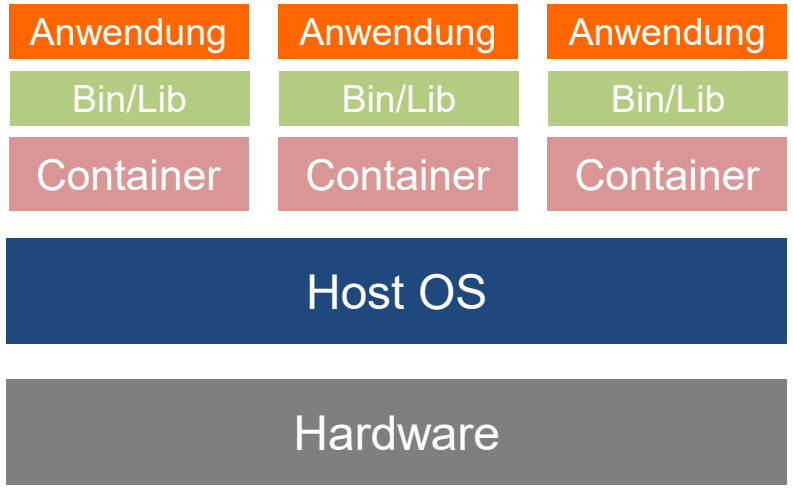
Agenda

1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	**Virtualisierung und Containerisierung**	**95**
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Definition

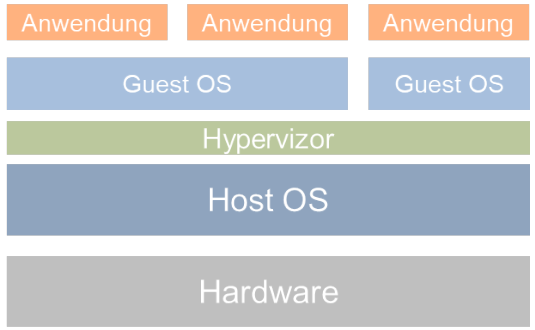


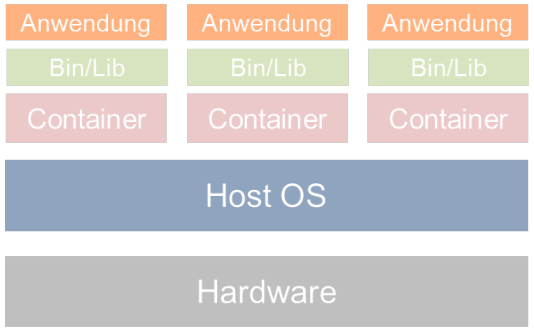
Virtualisierung ist die Abstraktion physischer Hardware- oder Software-Ressourcen zu einer virtuellen, vom Benutzer oder Programm als eigenständig wahrgenommenen Instanz, die von einer Virtualisierungsschicht bereitgestellt und verwaltet wird.



Containerisierung ist eine Virtualisierungstechnik auf Betriebssystemebene, bei der Anwendungen und ihre Abhängigkeiten in isolierten, standardisierten Umgebungen (Containern) laufen, die sich denselben Kernel teilen, aber voneinander und vom Host-System abgeschottet sind.

Vor- und Nachteile





Virtualisierung

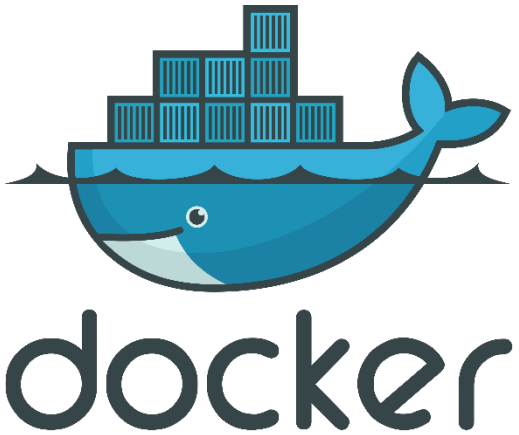
- ☺ Vollständige Isolation
- ☺ Erlaubt verschiedene Guest OS‘e
- ☹ Hoher Ressourcenverbrauch
- ☹ Startup in Minuten

Containerisierung

- ☺ Geringer Overhead, da Host OS gemeinsam genutzt wird
- ☺ Startup in Minuten
- ☹ Weniger strikte Isolation

Docker

Plattform für Containervirtualisierung, die Anwendungen und ihre Abhängigkeiten in leichtgewichtigen, portablen Containern verpackt und ausführt.



- ☺ Große Community und umfangreiches Ökosystem
- ☺ Open Source mit guter Dokumentation
- ☹ Daemon (Hintergrundprozess) benötigt standardmäßig Root-Rechte

Siehe: www.docker.com

Grundbegriffe

Image

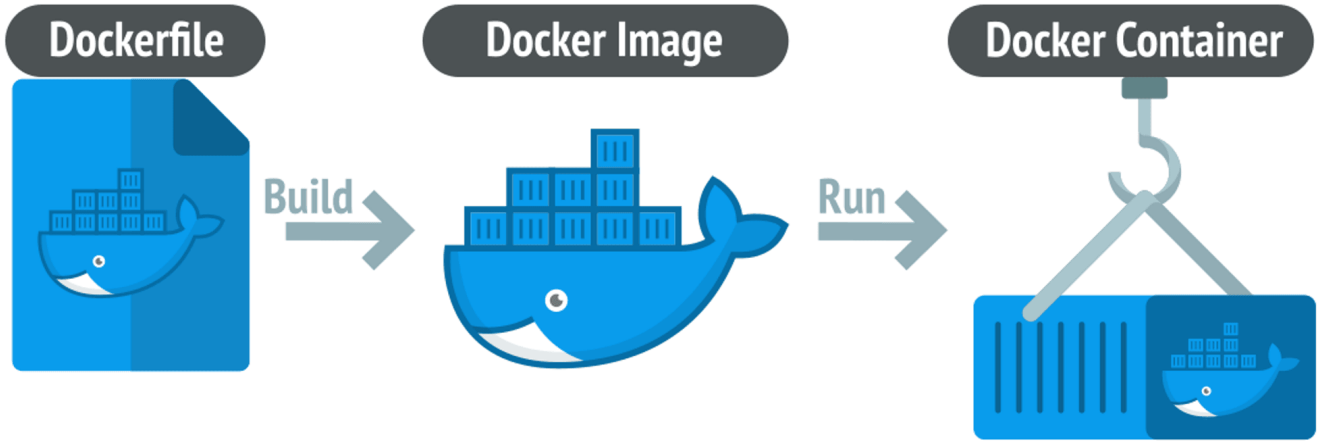
Ein unveränderliches Template, das die Konfiguration und die Software für einen Container definiert. Container werden aus Images gestartet.

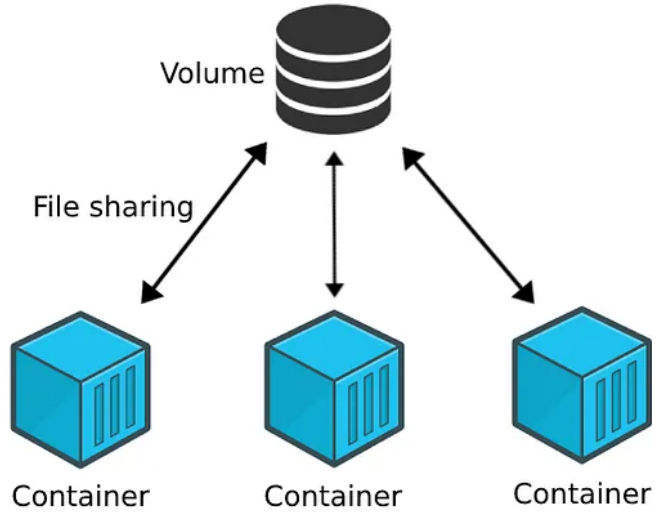
Container

Leichtgewichtige, isolierte Umgebung, die eine Anwendung und alle benötigten Abhängigkeiten enthält, während sie denselben Kernel wie das Host-Betriebssystem nutzt.

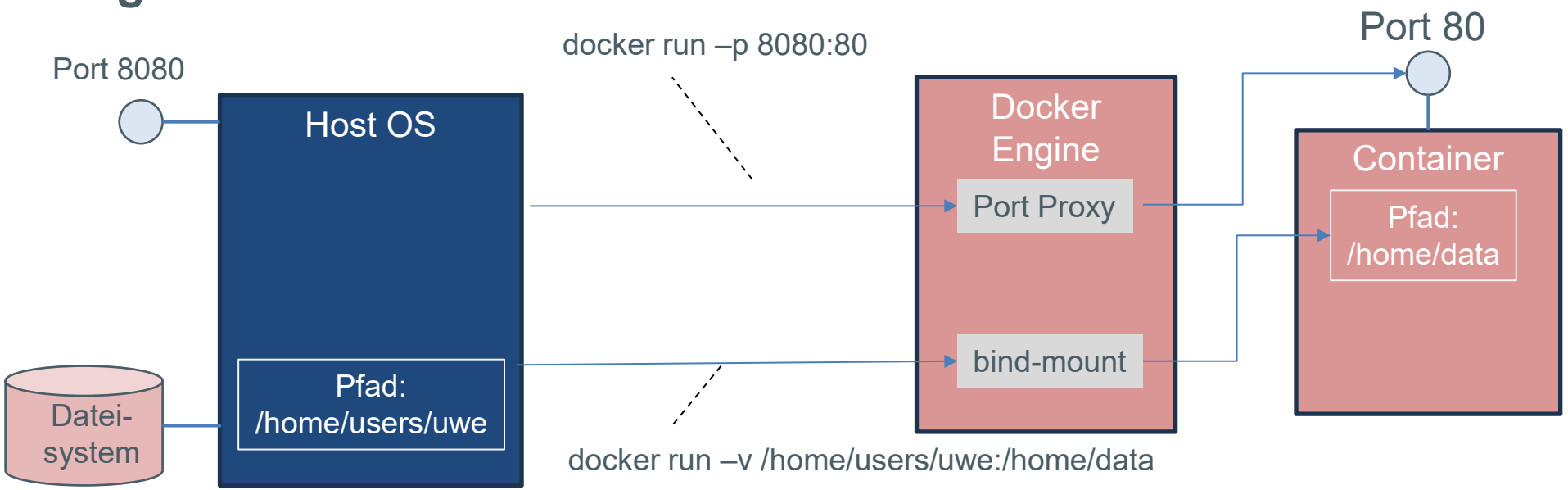
Volume

Ein Speichermechanismus, der Containern (von außen) bereitgestellten persistenten Speicherplatz ermöglicht.





Wie greift man auf Ressourcen des Host OS zu?



`docker run -p` : Publiziert einen (alle) Container-Port(s) auf dem Host

`docker run -v` : Stellt ein Verzeichnis des Hosts auf dem Container zur Verfügung (mount)

https://docs.docker.com/reference/cli/docker/container/run/

Docker File -> Build -> Run

